

ContactsFreeShare

A friendly, step-by-step guide to running the app — and your own private sharing server — on your own computer and Google account.

This handbook walks you through everything from start to finish. You do not need to be a programmer. Read it like a short book: the early chapters are easy, and the one technical chapter is broken into small, copy-and-paste steps you can follow slowly. Take your time, and don't skip the boxes labeled *Heads-up* and *Tip* — they save you the most trouble.

CONTENTS

1. [What this is, and what you'll need](#)
2. [Installing the desktop app](#)
3. [Connecting the app to your Google account](#)
4. [Setting up your own sharing server](#)
5. [Sharing contacts with other people](#)
6. [Keeping it running: updates, costs, shutdown](#)
7. [When something goes wrong](#)
8. [Quick reference](#)

What this is, and what you'll need

ContactsFreeShare is made of two separate pieces, and it helps to picture them before you begin.

The first piece is the **desktop app**. It runs entirely on your own Mac or Windows computer. You use it to view and edit your contacts and to keep them in sync with your Google account. Nothing about the desktop app is published to the internet — it is just a private window that opens in your web browser at an address that only your own computer can see.

The second piece is the **sharing server**. This is a small private website that you publish into *your own* Google Cloud account. Its only job is to let you share a group of contacts with other people — called **recipients** — and to quietly push your later edits out to them. You only need the server if you plan to share contacts. If you simply want to manage your own contacts, the desktop app alone is enough.

HEADS-UP: DIFFICULTY AND COST

Installing the desktop app (Chapters Two and Three) is easy — most people finish in about ten minutes.

Setting up the sharing server (Chapter Four) is the advanced part. It involves Google Cloud, a credit card for billing, and copying a few commands into a terminal window. Set aside one to two hours the first time, and if you've never used a terminal, it's perfectly fine to ask a more technical friend to sit with you for that chapter.

Because the server runs around the clock so it can deliver updates, Google Cloud charges a small monthly amount — often just a few dollars. You can shut it down at any time, and Chapter Six shows you how to watch and cap the cost.

What to have ready

For the desktop app you need a Google account — the one whose contacts you want to manage — and a computer running macOS or Windows. For the sharing server you additionally need a credit

or debit card to switch on Google Cloud billing, and a little patience for the one technical chapter.

You will also need two things from the person who gave you this guide: the **app download** (a `.zip` file, one for Mac and one for Windows), and — only if you're building the server — the **project source folder**, which is the folder that contains a sub-folder named `sharecontactsfree`.

Installing the desktop app

Start by getting the download link from the person who sent you this guide. There is a separate package for Mac and for Windows, so be sure to grab the right one for your computer.

On a Mac

Double-click the downloaded `.zip` file to unzip it. You'll get a folder called **ContactsFreeShare**. Open that folder and double-click **Start ContactsFreeShare.command** to launch the app.

The very first time, macOS may complain that the file is from an unidentified developer. If that happens, right-click the file, choose **Open**, and then click **Open** once more to confirm. If Python (a small piece of software the app needs) isn't already installed, the launcher downloads the installer and opens it for you; install it, then double-click **Start ContactsFreeShare.command** again.

If you'd like a tidy icon on your Desktop, open the folder once and double-click **Install Desktop Shortcut Mac.command**. Just keep the unzipped ContactsFreeShare folder where it is, because the shortcut starts the app from there.

On Windows

Right-click the downloaded `.zip` and choose **Extract All** to get a **ContactsFreeShare** folder. Open it and double-click **Start ContactsFreeShare**. If Windows shows a blue SmartScreen warning, click **More info** and then **Run anyway**. As on the Mac, if Python isn't installed the launcher fetches it for you; install it and start the app again. To add a Desktop icon, double-click **Install Desktop Shortcut Windows.bat** inside the folder.

TIP

When the app starts, your web browser automatically opens to `http://127.0.0.1:8000` . That address simply means "this very computer," so the app is private to you. If you ever close the browser tab, just reopen that address while the app is running.

Connecting the app to your Google account

Before the app can read or update your contacts, Google needs you to create a small permission slip called an **OAuth client**. It's free, and you only do it once.

TIP: DOING THE SERVER TOO?

If you also plan to set up the sharing server in Chapter Four, you can create *one* Google Cloud project and *one* OAuth client and use it for both. In that case, do steps one through five below, and later just add a couple more web addresses to the same client.

1. Go to console.cloud.google.com and sign in with your Google account.
2. Near the top, open the project dropdown and choose **New Project**. Give it a simple name like `my-contacts` and click **Create**.
3. Use the search bar to find **Google People API**, open it, and click **Enable**.
4. Search for **OAuth consent screen**. Choose **External**, click **Create**, then fill in an app name and your own email where asked (support email and developer contact). Click through the **Scopes** page without changes, and on the **Test users** page add your own Google email address.
5. Search for **Credentials**, click **Create Credentials**, and choose **OAuth client ID**. Pick **Web application** as the type. Under **Authorized redirect URIs**, click **Add URI** and paste this address exactly:

```
http://127.0.0.1:8000/google/connect/callback
```

Click **Create**. A small window shows your **Client ID** and **Client secret** — copy both somewhere safe.

Putting the keys into the app

The app keeps its settings in a small text file named `.env`, which it creates automatically the first time it runs. You'll find it here:

Mac	<code>~/Library/Application Support/ContactsFreeShare/.env</code>
Windows	<code>C:\Users\<your-name>\AppData\Roaming\ContactsFreeShare\.env</code>

TIP: FINDING THAT FOLDER

On a Mac, in Finder press **Cmd+Shift+G** and paste `~/Library/Application Support/ContactsFreeShare`. On Windows, type `%APPDATA%\ContactsFreeShare` into the File Explorer address bar and press Enter.

Open that `.env` file in a plain text editor — TextEdit on Mac or Notepad on Windows — and set these three lines, pasting in the values you copied a moment ago:

```
GOOGLE_OAUTH_CLIENT_ID=paste-your-client-id-here
GOOGLE_OAUTH_CLIENT_SECRET=paste-your-client-secret-here
GOOGLE_OAUTH_REDIRECT_URI=http://127.0.0.1:8000/google/connect/callback
```

Save the file, then fully quit and restart the app. Use the **Connect Google** option inside the app and sign in. Because this is a personal app you built yourself, Google shows an "unverified app" warning — that's expected. Click **Advanced**, then **Go to (your app name) (unsafe)**, and allow access.

That's it. You can now use the desktop app on its own. If you don't need to share contacts with other people, you can stop here. Continue to Chapter Four only if you want the sharing feature.

Setting up your own sharing server

This chapter publishes a tiny private website into your own Google Cloud account so you can share contact groups and push updates automatically. It is the longest chapter, so go slowly and finish each section before moving on.

Step 1 — Create a project and turn on billing

Sign in at console.cloud.google.com and either reuse the project from Chapter Three or create a new one. Make a note of its **Project ID** — it appears just under the project name and looks something like `my-contacts-123456`. Then use the search bar to open **Billing** and link a billing account by adding a card. The server needs billing switched on in order to run continuously.

TIP: CAP YOUR SPENDING

Under **Billing** → **Budgets & alerts** you can set a budget — say ten dollars a month — so Google emails you if costs ever climb. And you can delete the server entirely whenever you like (Chapter Six).

Step 2 — Install the Google Cloud tool (gcloud)

This is a small program that lets your computer talk to Google Cloud.

On a Mac, open the **Terminal** app and run:

```
curl https://sdk.cloud.google.com | bash
```

Then close and reopen Terminal. (If you prefer a clickable installer, follow the instructions at cloud.google.com/sdk/docs/install.)

On Windows, download and run the installer from that same page (look for the Windows installer). When it finishes, open the **Google Cloud SDK Shell** from the Start menu — use that

window for every command in this chapter. To confirm it worked, run:

```
gcloud --version
```

You should see version numbers rather than an error.

Step 3 — Sign in and choose your project

Run these one at a time, replacing `YOUR_PROJECT_ID` with the real ID you noted in Step 1:

```
gcloud auth login  
gcloud config set project YOUR_PROJECT_ID
```

The first command opens a browser so you can confirm — sign in with the same Google account.

Step 4 — Make sure you have an OAuth client

If you already created one in Chapter Three, you'll simply add two more web addresses to it later in Step 7 — there's nothing to do now. If you skipped Chapter Three, create one now: in the Console, enable the **Google People API**, set up the **OAuth consent screen** (External, with yourself added as a Test user), then under **Credentials** create an **OAuth client ID** of type **Web application** and copy its Client ID and Client secret.

Step 5 — Put the source code on your computer

You need the project's source folder — the one that contains the `sharecontactsfree` sub-folder. The person sharing this guide will give it to you as a `.zip` to unzip or a link to download. Put it somewhere easy to reach, such as your home folder, then move into it in the terminal. For example:

```
cd ~/PythonDB4-3-26
```

Use the actual name and location of the folder where you placed it.

Step 6 — Create the server's settings file

The server reads its settings from `sharecontactsfree/python/.env`. Create it from the included example:

```
cd sharecontactsfree/python
cp .env.example .env
```

Next, generate a random secret key. On Mac or Linux run the first command; on Windows (in the Google Cloud SDK Shell) run the second:

```
openssl rand -hex 32
```

```
python -c "import secrets; print(secrets.token_hex(32))"
```

Copy the long random string it prints. Then open `sharecontactsfree/python/.env` in a text editor and fill in these four lines — use that same random string for both `APP_SECRET_KEY` and `CFS_API_KEY`:

```
GOOGLE_CLIENT_ID=paste-your-client-id-here
GOOGLE_CLIENT_SECRET=paste-your-client-secret-here
APP_SECRET_KEY=paste-the-random-string-you-generated
CFS_API_KEY=paste-the-same-random-string-again
```

HEADS-UP

Keep `APP_SECRET_KEY` safe and don't change it later — changing it logs out everyone you've shared with. Reusing the same value for `CFS_API_KEY` is what lets your desktop app send updates to the server. Leave the rest of the file as it is; the deploy script fills in everything else automatically.

Step 7 — Publish (deploy) the server

From the main project folder, run the deploy script:

```
cd ../..
bash sharecontactsfree/scripts/deploy_cloud_run.sh
```

The first run takes several minutes. It switches on the Google Cloud services it needs, creates a storage area for the database, and builds and publishes your server. If it asks you to confirm enabling anything, say **yes**. When it finishes, it prints something like this:

```
Deployed: https://share-contacts-xxxxxxx-uc.a.run.app
```

Copy that `https://...run.app` address — you'll use it a few times below.

Step 8 — Tell Google about the server's web addresses

In the Console, go to **Credentials** and click your Web-application OAuth client. Under **Authorized redirect URIs**, add both of these, replacing the address with *your* real one from Step 7, then click **Save**:

```
https://share-contacts-xxxxxxx-uc.a.run.app/auth/owner/callback  
https://share-contacts-xxxxxxx-uc.a.run.app/auth/recipient/callback
```

If this is the same client your desktop app uses, leave the earlier

```
http://127.0.0.1:8000/google/connect/callback
```

 entry in place as well.

Step 9 — Make recipient logins reliable

While your consent screen is in **Testing** mode, Google expires permission after seven days, which would quietly break automatic updates to the people you share with. The simplest fix for friends and family is to publish the app: open the **OAuth consent screen**, click **Publish App**, and confirm to move it to **In production**. You can do this without formal Google verification — recipients just click past an "unverified app" notice — and it removes the seven-day limit. There's a cap of about a hundred users, which is far more than enough for personal use.

If you'd rather stay in Testing mode, you instead have to add every recipient's Google email as a Test user, and re-approve permissions every seven days.

Step 10 — Point your desktop app at your server

Open the desktop app and go to **App Settings**. Set **Share web app URL** to your server address from Step 7, and set **Share web API key** to the same random string you used for

`APP_SECRET_KEY` in Step 6. Save, and your desktop app is now wired to your very own sharing server.

Sharing contacts with other people

With the server running, sharing is straightforward. In your browser, open your server's owner page — your `https://...run.app/` address — and sign in with Google. Choose a contact group to share and enter a recipient's email address.

Then send that recipient the invite link below. They open it in any web browser; they do *not* install anything:

```
https://share-contacts-xxxxxxx-uc.a.run.app/?mode=shared
```

The recipient signs in with their own Google account and clicks **Connect account**. Your server then imports the shared contacts into their Google account in the background. Later, whenever you edit those contacts in the desktop app and upload your changes, the server quietly pushes the updates out to everyone you've shared with.

Keeping it running: updates, costs, and shutdown

When you receive a newer version of the source code, you can update the server simply by running the deploy script again:

```
bash sharecontactsfree/scripts/deploy_cloud_run.sh
```

To keep an eye on cost, visit **Billing** → **Budgets & alerts** in the Google Cloud Console and set a small monthly budget so you're emailed if spending grows.

To pause or stop charges entirely, open **Cloud Run** in the Console, click the **share-contacts** service, and choose **Delete**; you can always redeploy later. Lowering the minimum instances to zero also reduces cost, though background updates to recipients become less reliable when the server is allowed to sleep.

When something goes wrong

A few problems come up often enough to list here, along with their fixes.

If the terminal says "**gcloud: command not found**," the Google Cloud tool either isn't installed or you didn't reopen the terminal after installing it. Redo Step 2 of Chapter Four and open a fresh terminal window.

If the deploy script complains that **APP_SECRET_KEY is required**, you haven't filled in `sharecontactsfree/python/.env` — revisit Step 6. If it says **GOOGLE_CLIENT_ID is required**, the Client ID or Secret lines in that file are still blank; paste them from your OAuth client.

If Google sign-in shows "**redirect_uri_mismatch**," the exact web address isn't listed in your OAuth client's Authorized redirect URIs. Re-check Chapter Three's step five and Chapter Four's step eight; the address must match character for character, including `http` versus `https` and any trailing slash.

An "**unverified app**" warning is normal for a personal build — click **Advanced** and continue. If recipients stop syncing after about a week, your consent screen is still in Testing mode; publish it to production as described in Chapter Four, step nine. And if the desktop app's **Connect Google** fails, double-check the three `GOOGLE_OAUTH_*` lines in the desktop `.env` file and make sure you restarted the app afterward.

Quick reference

Keep this table handy — it collects every address and file path used in the guide.

What	Value
Desktop app address	<code>http://127.0.0.1:8000</code>
Desktop settings file (Mac)	<code>~/Library/Application Support/ContactsFreeShare/.env</code>
Desktop settings file (Windows)	<code>%APPDATA%\ContactsFreeShare\.env</code>
Desktop redirect address	<code>http://127.0.0.1:8000/google/connect/callback</code>
Server settings file	<code>sharecontactsfree/python/.env</code>
Server redirect addresses	<code><server-url>/auth/owner/callback</code> <code><server-url>/auth/recipient/callback</code>
Server owner page	<code><server-url>/</code>
Recipient invite link	<code><server-url>/?mode=shared</code>
Share web API key (in desktop)	same as <code>APP_SECRET_KEY</code> / <code>CFS_API_KEY</code>

DON'T WANT TO RUN A SERVER AT ALL?

A friend doesn't have to deal with Google Cloud. They can use the desktop app on its own to manage their contacts, and whoever already runs a sharing server can simply add them as another *owner* — the server is built to host many different owners at once.

You did it. Keep this guide somewhere handy in case you ever need to reinstall, update, or help a friend through the same steps.